Milestone 2: Environment Setup

Project Title: Emergent Defensive Strategies in Multi-Agent Hide & Seek

Name: Justin Wiseman (jwisema8)

Course: ECE 414 - Introduction to Reinforcement Learning

1. Project Overview

- **What is your project idea?**
  I am designing a custom 2D Gymnasium grid-world (HideAndSeek-v0) where a "Hider" agent learns to survive against an adversarial "Seeker" by manipulating objects in the environment. During a preparation phase, the Hider can grab, carry, and lock blocks into place to construct defensive barricades before the Seeker is released.
- **What have you implemented so far?**
  I have implemented the complete custom Gymnasium environment (hidenseek_env.py), including the Dict observation space, the Discrete(8) action space, collision detection, Pygame rendering, and the two-phase episode structure (Preparation and Chase). I also implemented a Breadth-First Search (BFS) reachability check to detect when the Hider has completely barricaded itself from the Seeker, as well as a manual keyboard control script (manual_control.py) for testing.
- **What remains to be completed?**
  Next, I need to build the tabular Q-Learning and SARSA training loops for the Hider agent and refine the state representation so the table remains tractable. Afterward, I will evaluate the trained agent against random and rule-based baselines.

2. Environment Design

- **Environment:** A 10×10 2D discrete grid-world rendered in Pygame containing boundary walls, one Hider (blue circle), one Seeker (red circle), and three movable blocks (brown when free, yellow when grabbed, dark gray when locked).
- **State/Observation:** A spaces.Dict containing "hider" (x, y) coordinates (MultiDiscrete([10, 10])), "seeker" (x, y) coordinates (MultiDiscrete([10, 10])), "phase" (Discrete(2) where 0 is Prep and 1 is Chase), and "blocks" (MultiDiscrete tracking $[x, y, is_grabbed, is_locked]$ for each block).
- **Action:** A Discrete(8) space representing: Move Up (0), Move Down (1), Move Left (2), Move Right (3), Grab Block (4), Drop Block (5), Lock Block (6), and Unlock Block (7).
- **Reward:** A sparse reward structure where the Hider receives a -0.01 step penalty to encourage efficiency, +1.0 per step survived during the Chase phase, -10.0 if caught by the Seeker, and +100.0 (plus remaining survival step points) for either surviving until the 200-step limit or completely blocking all valid Seeker paths via barricade.

- **Transition:** Directional actions update the Hider's coordinates if the target cell is within grid bounds and not occupied by a locked block; if a block is currently grabbed, its coordinates move with the Hider. After 50 steps (prep_phase_steps), the environment transitions to the Chase phase (phase = 1), releasing a greedy Seeker that steps toward the Hider each turn while respecting locked block collisions.

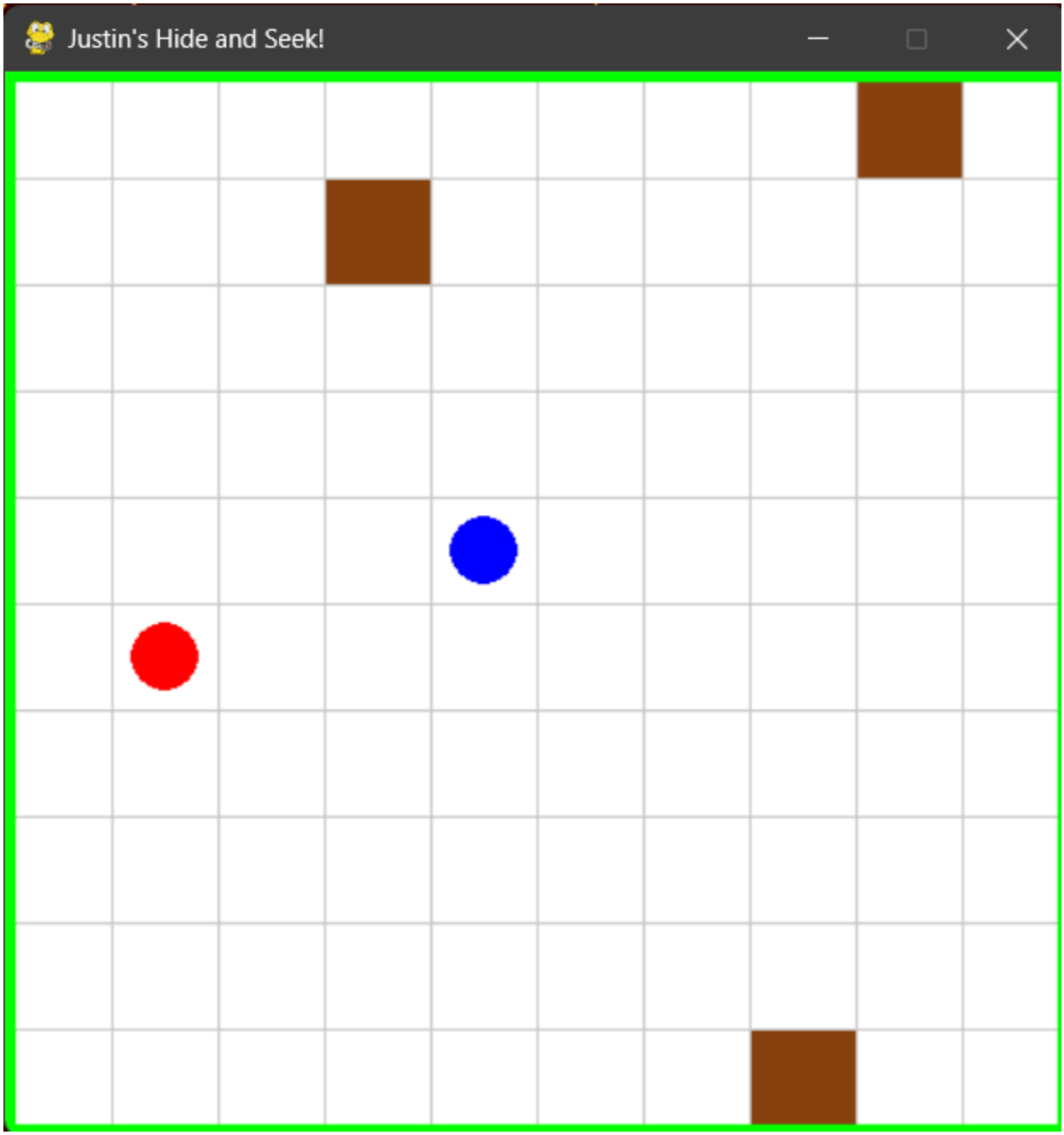


Figure 1: Initial state of the custom HideAndSeek-v0 environment in Pygame during the Preparation Phase (indicated by the green border).

```python
def step(self, action):
    self.current_step += 1
    reward = -0.01 # Base penalty.

    # Hider Movement.
    if action < 4:
        moves = {
            0: np.array([0, -1]),  # Up
            1: np.array([-1, 0]),   # Left
            2: np.array([0, 1]),  # Down
            3: np.array([1, 0])    # Right
        }
        target_pos = self.hider_pos + moves[action]

        if self._is_valid_position(target_pos):
            self.hider_pos = target_pos

            for block in self.blocks:
                if block[2] == 1: # Grabbed
                    block[0], block[1] = self.hider_pos[0], self.hider_pos[1]

    # Hider Interactions.
        # Grab.
    elif action == 4:
        if not any(b[2] == 1 for b in self.blocks):
            for block in self.blocks:
                if np.array_equal(self.hider_pos, block[:2]) and block[3] == 0:
                    block[2] = 1
                    break

        # Drop.
    elif action == 5:
        for block in self.blocks:
            if block[2] == 1:
                block[2] = 0

        # Lock.
    elif action == 6:
        for block in self.blocks:
            if block[2] == 1:
                block[3] = 1
                block[2] = 0
```

```python
    # Unlock.
elif action == 7:
    for block in self.blocks:
        if block[3] == 1:
            dist = np.linalg.norm(self.hider_pos - block[:2])
            if dist <= 1.0:
                block[3] = 0

# Phase Management & Seeker Logic
if self.current_step > self.prep_phase_steps:
    self.phase = 1
    reward += 1.0 # Reward for each step in chase.

    seeker_move = np.zeros(2, dtype=int)
    if self.seeker_pos[0] < self.hider_pos[0]:
        seeker_move[0] = 1
    elif self.seeker_pos[0] > self.hider_pos[0]: seeker_move[0] = -1
    elif self.seeker_pos[1] < self.hider_pos[1]: seeker_move[1] = 1
    elif self.seeker_pos[1] > self.hider_pos[1]: seeker_move[1] = -1

    seeker_target = self.seeker_pos + seeker_move
    if self._is_valid_position(seeker_target):
        self.seeker_pos = seeker_target

# Termination & Rewards
terminated = False
truncated = False

    # 1. Check if Seeker caught Hider
if np.array_equal(self.hider_pos, self.seeker_pos) and self.phase == 1:
    terminated = True
    reward -= 10.0 # Caught penalty
    print("Seeker caught the Hider! Episode Over.")

    # 2. Check if Hider successfully barricaded themselves (No valid path)
elif not self._has_path_to_hider():
    terminated = True
    # Award the +100 win bonus PLUS the remaining survival step points
    # so the RL agent isn't incentivized to stall before locking the final block
    remaining_steps = max(0, 200 - self.current_step)
    reward += 100.0 + remaining_steps
    print("No valid path for Seeker! Hider wins by barricade!")
```

```python
    # 3. Check if max steps reached
elif self.current_step >= 200:
    truncated = True
    reward += 100.0 # Survival bonus
    print("Max steps reached! Hider survives!")

return self._get_obs(), reward, terminated, truncated, self._get_info()
```

```python
def _has_path_to_hider(self):
    # If the Hider is still standing on top of a locked block, wait until they step off
    if not self._is_valid_position(self.hider_pos):
        return True

    start = tuple(self.seeker_pos)
    target = tuple(self.hider_pos)

    queue = deque([start])
    visited = {start}
    directions = [(0, -1), (0, 1), (-1, 0), (1, 0)]

    while queue:
        curr_x, curr_y = queue.popleft()
        if (curr_x, curr_y) == target:
            return True

        for dx, dy in directions:
            next_pos = np.array([curr_x + dx, curr_y + dy])
            next_tuple = (int(next_pos[0]), int(next_pos[1]))

            if next_tuple not in visited and self._is_valid_position(next_pos):
                visited.add(next_tuple)
                queue.append(next_tuple)

    return False
```

Figure 2: Implementation of the step() transition logic and BFS barricade verification in hidenseek_env.py, handling movement collisions, block grabbing/locking, phase transitions, and termination checks.

3. Environment Testing

- **How did you test your environment?**
  I built an interactive Pygame keyboard testing script (scripts/manual_control.py) that maps W, A, S, D to movement and G, H, L, U to Grab, Drop, Lock, and Unlock actions. This allowed me to manually playtest edge cases, wall collisions, and phase transitions in real time.

- **What happens when the agent performs an action?**
  Moving onto an unlocked brown block and pressing G attaches the block to the Hider (turning it yellow) so it moves alongside the agent. Pressing L locks the block in place (turning it dark gray) and drops it, turning that grid cell into an impassable obstacle for both the Hider and Seeker.
- **Does reset() correctly initialize the environment?**
  Yes, reset() uses Gymnasium's seeded random number generator (self.np_random) to spawn the Hider, Seeker, and blocks on non-overlapping tiles, resets all block grab/lock flags to 0, resets the step counter, and returns the environment to the Preparation phase (phase = 0).
- **What technical challenges or constraints have you implemented?**
  I implemented the phased temporal constraint (Seeker stasis for the first 50 steps) supporting the sparse reward/credit assignment challenge, physical block collision mechanics, and a BFS pathfinding verifier (_has_path_to_hider()) that detects when a locked barricade eliminates all valid paths between the Seeker and Hider.

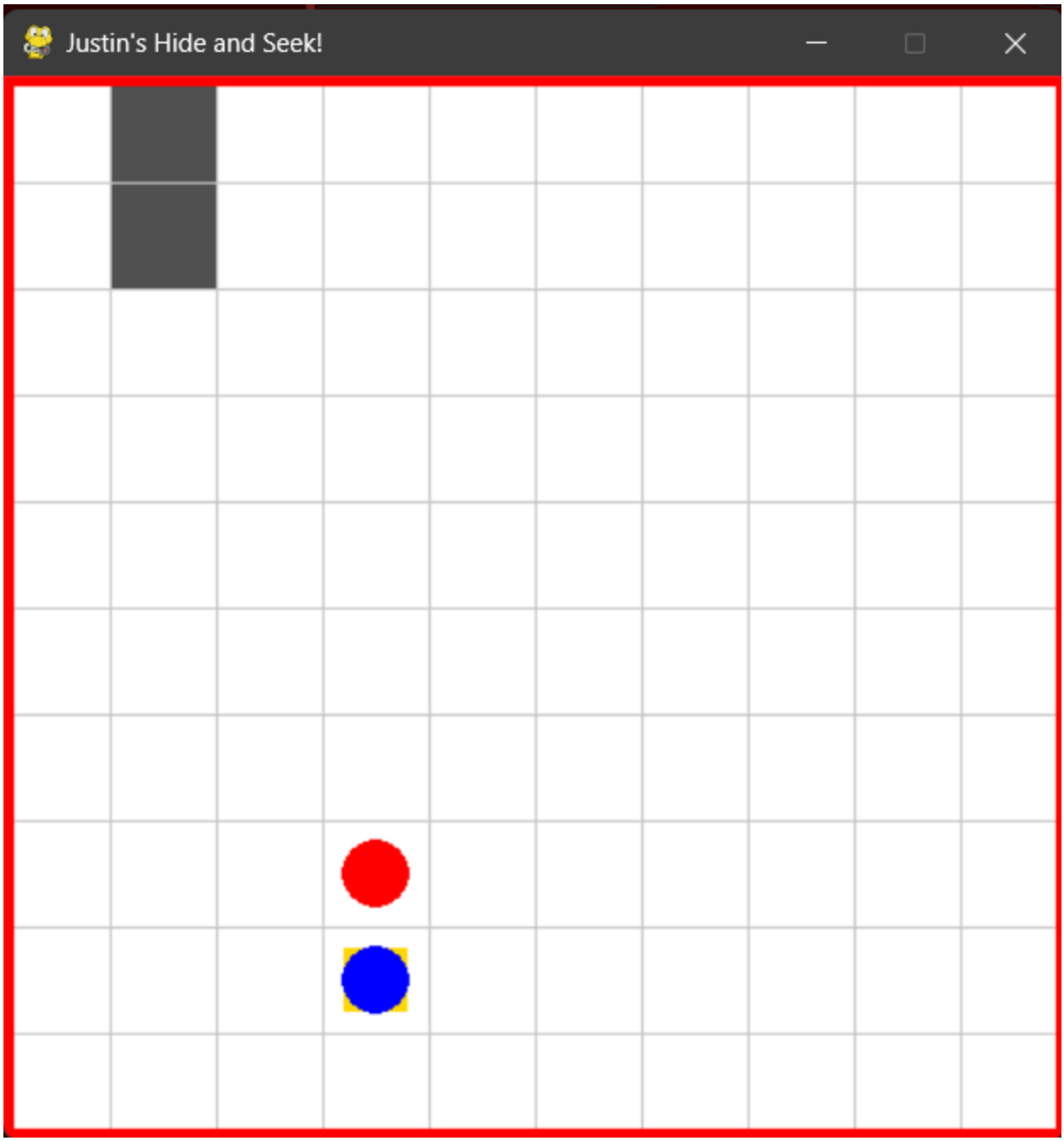


Figure 3: Execution test demonstrating the Hider grabbing blocks (yellow), locking them into a defensive corner barricade (dark gray), and triggering the BFS unreachable-path win condition.

4. Next Steps

- **What will you implement next?**
  I will implement the tabular Q-Learning and SARSA agents for the Hider alongside a

state-aggregation/discretization wrapper (such as relative coordinates) so the Q-table remains compact enough to converge rapidly.

- **What problems are you currently facing?**
  The raw combinatorial state space of a 10×10 grid with multiple movable blocks is very large for standard tabular Q-Learning. To prevent slow convergence, I will need to either reduce the grid size during initial tabular training or encode relative positions before scaling up.